

PROGRAMAÇÃO MODULAR

RELACIONAMENTOS ENTRE
CLASSES: ASSOCIAÇÃO,
AGREGAÇÃO E COMPOSIÇÃO

PROF. JOÃO CARAM

O MUNDO E OBJETOS COMPOSTOS

2

- No nosso dia a dia, utilizamos objetos.
- Frequentemente, utilizamos objetos compostos.

O MUNDO E OBJETOS COMPOSTOS

3

- Um computador é composto por diversos componentes: processador, memória principal, discos de armazenamento, tela/monitor, portas de saída, dispositivos de entrada.
- Existem “famílias” de computadores.
- E também existem “famílias” de componentes.

P OO E O BJETOS

4

- Coleção de objetos: um catálogo de componentes reutilizáveis.
- Reúso é fundamental para reduzir custos e aumentar a confiabilidade.
 - Modularidade é a chave para atingir alto grau de reúso.

OBJETOS E RELACIONAMENTOS

5

- Em geral, objetos não funcionam sozinhos:
 - Usam/se comunicam com outros;
 - Contêm ou são formados por outros;
- P00, segundo Alan Kay:
 - troca de mensagens;
 - proteção e retenção local e ocultamento do estado;
 - associação tardia e dinâmica de tudo o que for possível.

OBJETOS E RELACIONAMENTOS

6

- ▣ P00, segundo Alan Kay:
 - ▣ troca de mensagens.
- ▣ Projeto de um sistema modular com OO:
 - ▣ Classes/objetos;
 - ▣ Relacionamento/comunicação entre elas;

7

ASSOCIAÇÃO



ASSOCIAÇÃO

8

- Relacionamento denotado por “usa um”.
- Objetos são associados, mas não há relação de pertinência.
 - Ex: Um controle remoto comanda uma televisão.
Pessoas jantam em restaurantes.
Computador está conectado ao projetor.

ASSOCIAÇÃO

9

- ▣ Relacionamento denotado por “usa um”.
- ▣ Objetos são associados, mas não há relação de pertinência.
 - ▣ Ex: Um controle remoto comanda uma televisão.
Pessoas jantam em restaurantes.
Computador está conectado ao projetor.

ASSOCIAÇÃO

UML: linha simples

ControleRemoto
-aparelho: Televisao
+ControleRemoto() +associarTV(tv:Televisao): void +mudarCanal(canal:int): int +aumentarVolume(): int +diminuirVolume(): int

* controla 1

Televisao
-VOL_MAXIMO: int = 100 -canal: int -canalAnterior: int -ligada: boolean -volume: int +Televisao() +ligar(): void +desligar(): void +mudarCanal(canal:int): int +canalAnterior(): int +aumentarVolume(): int +diminuirVolume(): int

ASSOCIAÇÃO

```
public class ControleRemoto{  
    private Televisao aparelho;  
    ...  
    public void associarTV(Televisao tv){  
        aparelho = tv;  
    }  
    ...  
    public int mudarCanal(int canal){  
        int resposta = -1;  
        if(aparelho != null)  
            resposta = aparelho.mudarCanal(canal);  
        return resposta;  
    }  
}
```

ASSOCIAÇÃO

```
public class ControleRemoto{  
    private Televisao aparelho;  
    ...  
    public void associarTV(Televisao tv){  
        aparelho = tv;  
    }  
    ...  
    public int mudarCanal(int canal){  
        int resposta = -1;  
        if(aparelho != null)  
            resposta = aparelho.mudarCanal(canal);  
        return resposta;  
    }  
}
```

13

AGREGAÇÃO



AGREGAÇÃO

14

- Indica um relacionamento do tipo todo/parte, ou “tem/contém um”.
- O objeto é definido em termos de seus componentes.

AGREGAÇÃO

15

- ▣ Objeto definido em termos dos seus componentes:
 - ▣ Ex: Automóvel contém um motor e portas.
Um departamento abriga professores.
Inventário do personagem contém equipamentos.

AGREGAÇÃO

16

- ▣ Objeto definido em termos dos seus componentes:
 - ▣ Ex: Automóvel contém um motor e portas.
Um departamento abriga professores.
Um time tem atletas.
Inventário do personagem contém equipamentos.

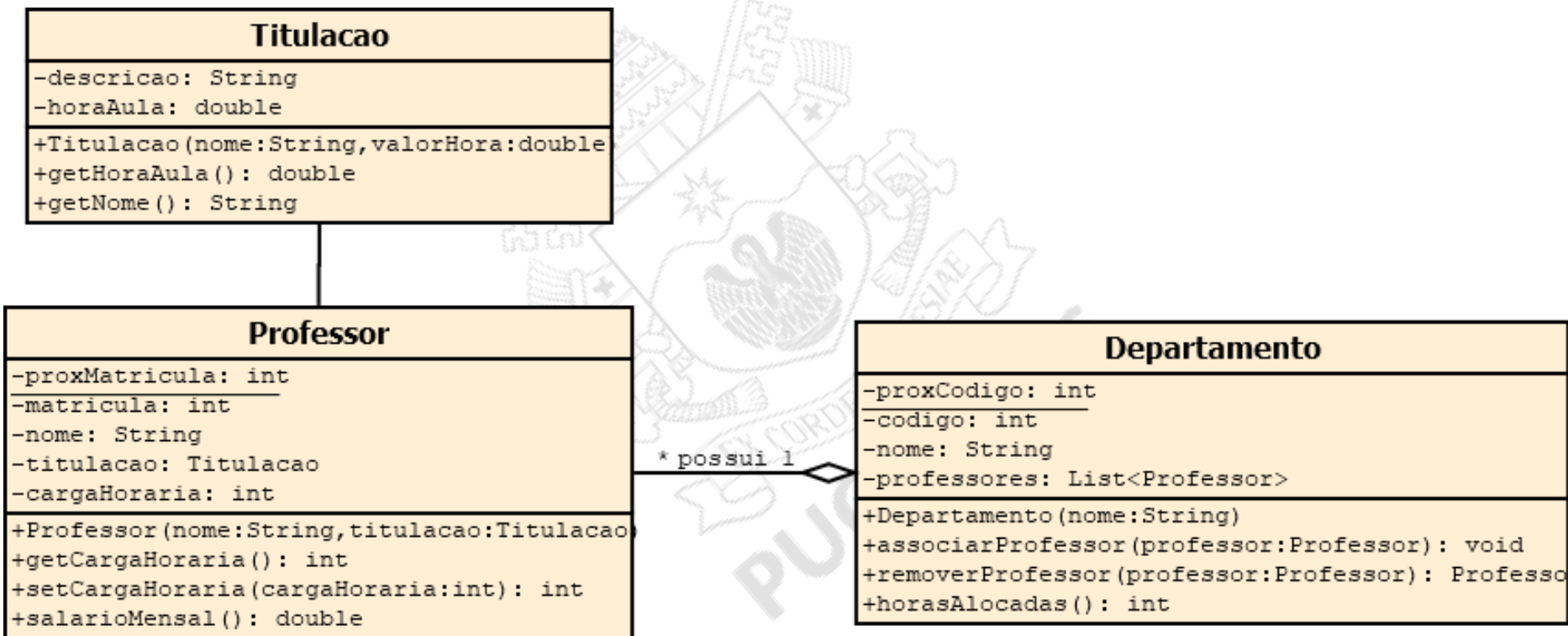
AGREGAÇÃO

17

- ▣ Tempos de vida independentes:
 - ▣ a existência da “parte” faz sentido, mesmo não existindo “todo”.
 - ▣ Ex: professor → departamento
atletas → time

AGREGAÇÃO

Representação gráfica: losango (no lado todo)



AGREGAÇÃO

```
public class Departamento{  
    private static int proxCodigo = 1;  
    private int codigo;  
    private String nome;  
    private List<Professor> professores;  
  
    public Departamento(String nome){  
        codigo = proxCodigo;  
        proxCodigo++;  
        professores = new LinkedList<>();  
    }  
}
```

AGREGAÇÃO

```
public class Departamento{  
    ...  
    public int horasAlocadas(){  
        int total = 0;  
        for(Professor prof : professores)  
            total += prof.getCargaHoraria();  
        return total;  
    }  
    ...  
}
```

AGREGAÇÃO

```
public class Departamento{  
    ...  
    public int horasAlocadas(){  
        int total = 0;  
        for(Professor prof : professores)  
            total += prof.getCargaHoraria();  
        return total;  
    }  
    ...  
}
```

COMPOSIÇÃO



COMPOSIÇÃO

23

- Um objeto “é formado por” outros objetos:
 - relação “está contido”;
 - dependência do tempo de vida entre parte e todo.
- Relacionamento mais forte que a agregação.
 - Acoplamento . . .

COMPOSIÇÃO

24

- Um objeto “é formado por” outros objetos:
 - Ex: Um livro é formado por vários capítulos.
Um banco controla várias contas.
Um pedido é formado por vários itens.

COMPOSIÇÃO

25

- ▣ Um objeto “é formado por” outros objetos.
 - ▣ Ex: Um **livro** é formado por vários **capítulos**.
Um **banco** controla várias **contas**.
Um **pedido** é formado por vários **itens**.
- ▣ A existência da parte **NÃO** faz sentido se o todo não continuar existindo.

COMPOSIÇÃO

Representação gráfica: losango preenchido (no lado todo)

Agencia
-nome: String -codigo: int -contas: Map<Integer, Conta>
+Agencia(nome:String,codigo:int) +associarConta(conta:Conta): void +removerConta(conta:Conta): Conta +localizarConta(codigoConta:int): Conta +valorEmCustodia(): double



Conta
-cpfTitular: String -codigo: int -saldo: double
+Conta(cpf:String,codigo:int) +sacar(valor:double): boolean +depositar(valor:double): boolean +getSaldo(): double

COMPOSIÇÃO

```
public class Agencia{  
    private String nome;  
    private int codigo;  
    private Map<Integer, Conta> contas;  
  
    public Agencia(String nome, int codigo){  
        this.nome = nome;  
        if(codigo < 1)  
            codigo = 1;  
        this.codigo = codigo;  
        contas = new HashMap<>(1000);  
    }  
}
```

COMPOSIÇÃO

```
public class Agencia{  
    public Conta localizarConta(int codigoConta){  
        return contas.get(codigoConta);  
    }  
    ...  
    public double valorEmCustodia(){  
        double valor = 0d;  
        for(Conta conta : contas.values())  
            valor += conta.getSaldo();  
        return valor;  
    }  
}
```

AGREGAÇÃO X COMPOSIÇÃO

29

- Relacionamentos do tipo composição têm implicações na criação e finalização dos objetos:
 - Criar objetos somente com a relação estabelecida.
 - Apagados os objetos associados na finalização.
- A diferença de entendimento entre agregação e composição é sutil e pode variar com o contexto.

AGREGAÇÃO X COMPOSIÇÃO

30

- A diferença de entendimento entre agregação e composição é sutil e pode variar com o contexto.
- Turmas de disciplinas e horários.
- Inventário de personagem e equipamentos.

IMPLEMENTAÇÃO DE RELAÇÕES



RELACIONAMENTOS ENTRE CLASSES

32

- Associação, agregação, composição:
 - conceitos similares (objeto usa outro);
 - comportamentos diferentes.
- Implementação, em geral:
 - Referências de objetos.
 - Delegação de responsabilidade.

IMPLEMENTAÇÃO DE RELACIONAMENTOS

33

- Pergunta fundamental:
 - Unidirecional ou bidirecional?
- Unidirecionais sugerem a implementação de atributos apenas na origem.
 - Alguns casos exigem análise mais aprofundada.
- Atenção para o acoplamento.

IMPLEMENTAÇÃO DE RELACIONAMENTOS

ControleRemoto

-aparelho: Televisao

+ControleRemoto()

+associarTV(tv:Televisao): void

+mudarCanal(canal:int): int

+aumentarVolume(): int

+diminuirVolume(): int

* controla 1

Televisao

-VOL_MAXIMO: int = 100

-canal: int

-canalAnterior: int

-ligada: boolean

-volume: int

+Televisao()

+ligar(): void

+desligar(): void

+mudarCanal(canal:int): int

+canalAnterior(): int

+aumentarVolume(): int

+diminuirVolume(): int

IMPLEMENTAÇÃO DE RELACIONAMENTOS

ControleRemoto

-aparelho: Televisao

+ControleRemoto()

+associarTV(tv:Televisao): void

+mudarCanal(canal:int): int

+aumentarVolume(): int

+diminuirVolume(): int

* controla 1

Televisao

-VOL_MAXIMO: int = 100

-canal: int

-canalAnterior: int

-ligada: boolean

-volume: int

+Televisao()

+ligar(): void

+desligar(): void

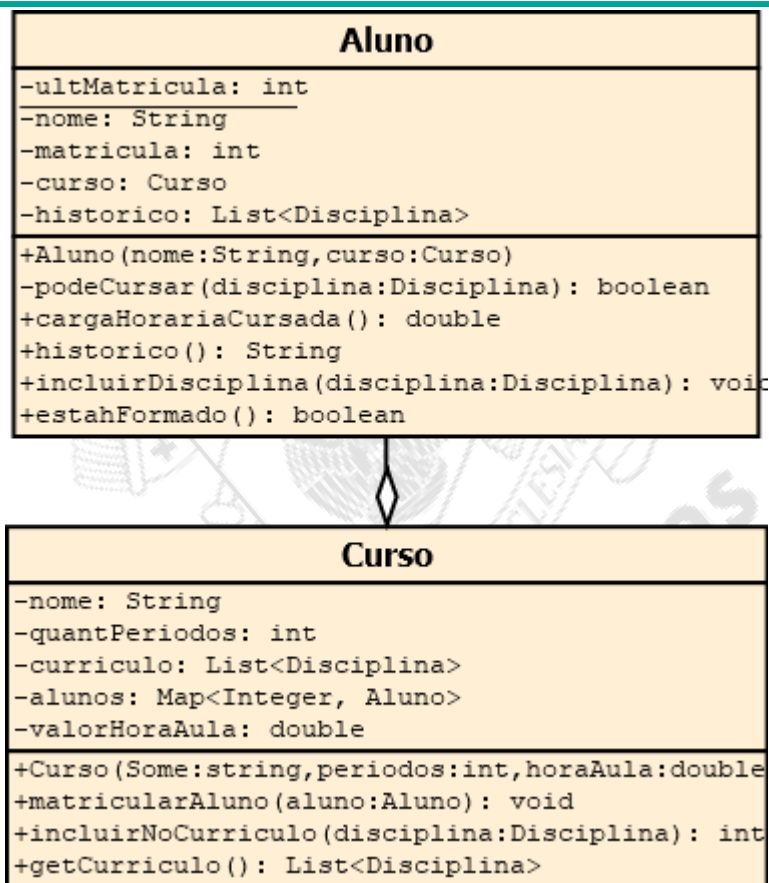
+mudarCanal(canal:int): int

+canalAnterior(): int

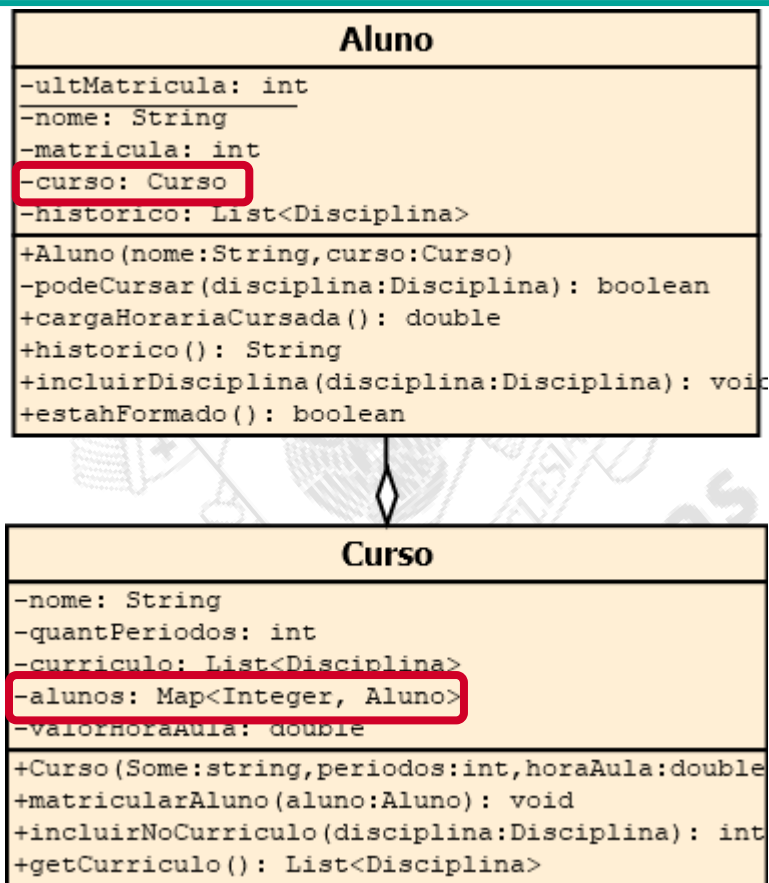
+aumentarVolume(): int

+diminuirVolume(): int

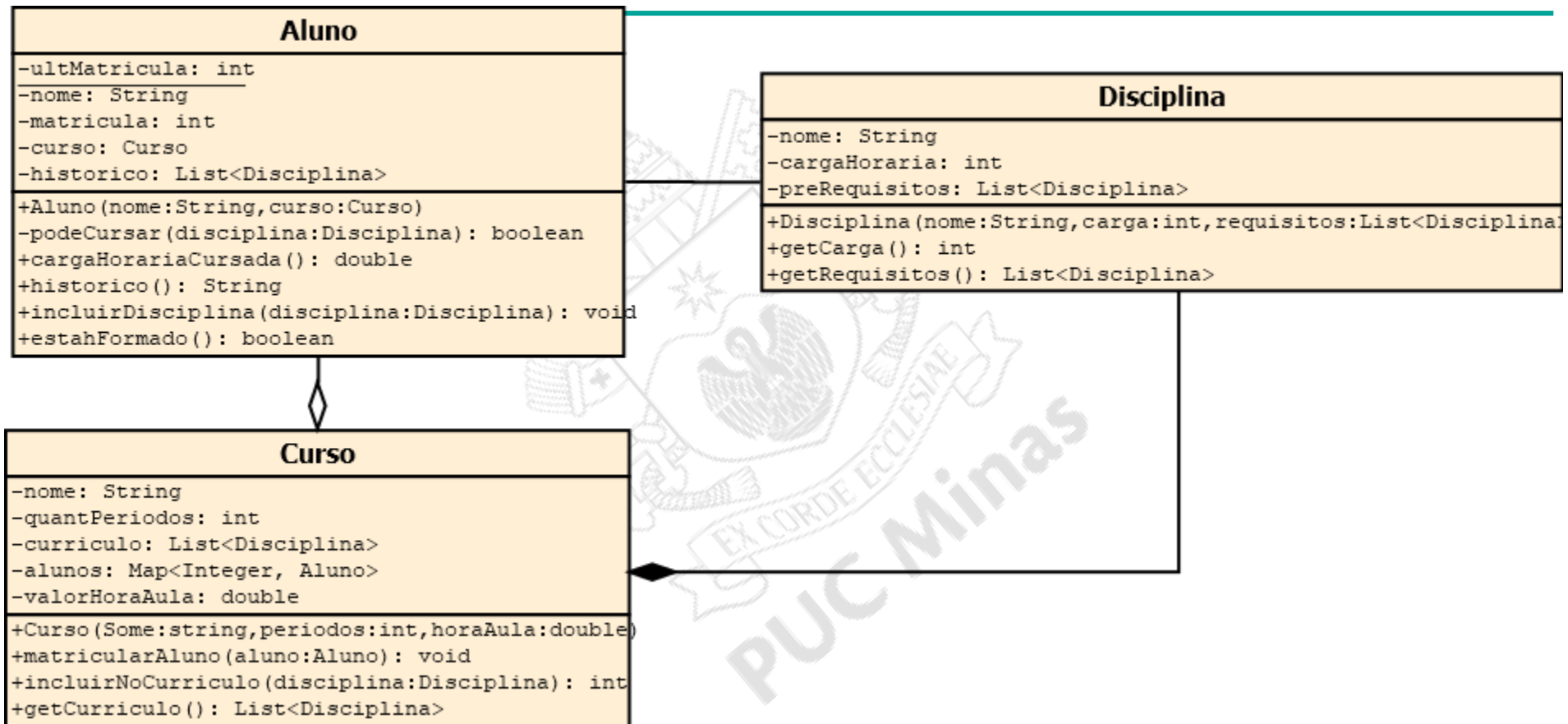
IMPLEMENTAÇÃO DE RELACIONAMENTOS



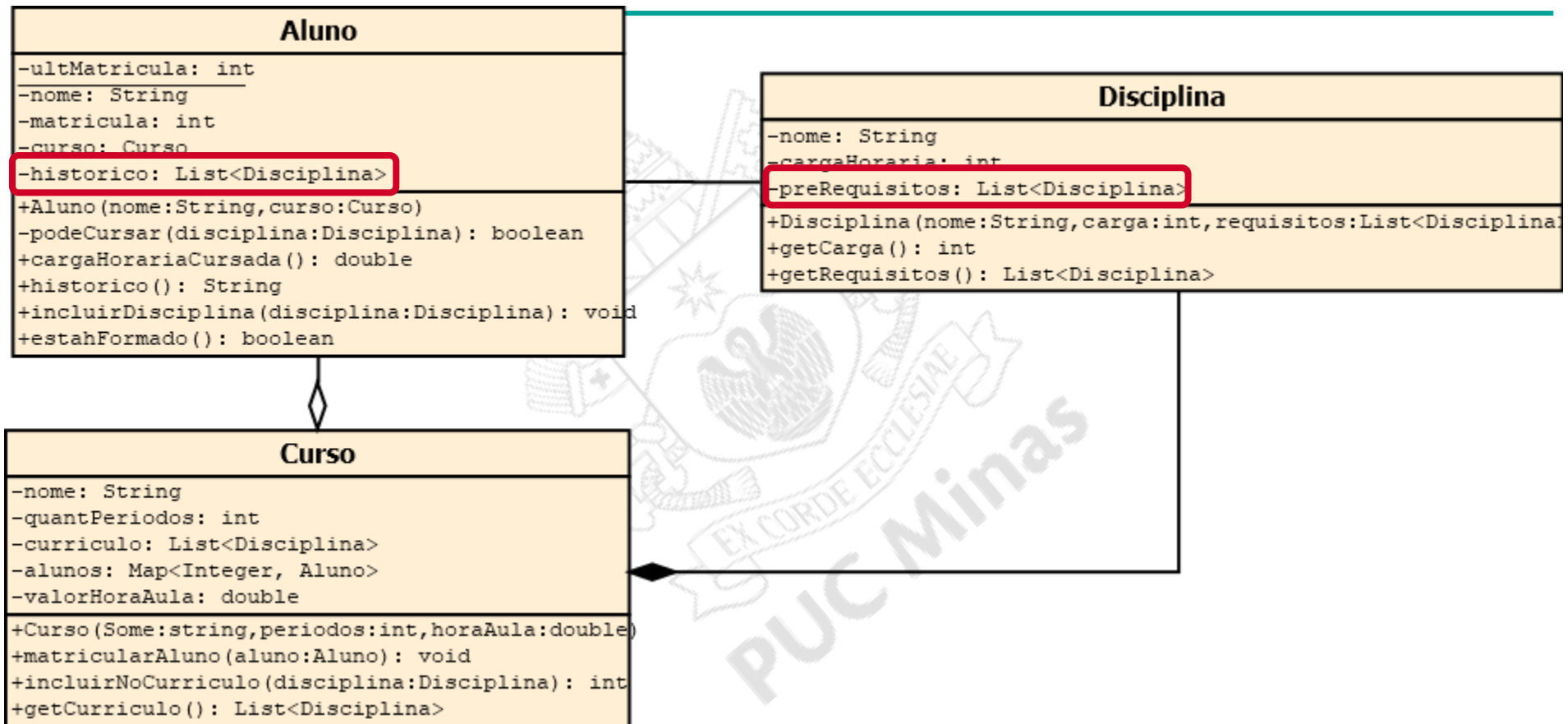
IMPLEMENTAÇÃO DE RELACIONAMENTOS



IMPLEMENTAÇÃO DE RELACIONAMENTOS



IMPLEMENTAÇÃO DE RELACIONAMENTOS



E O SISTEMA?

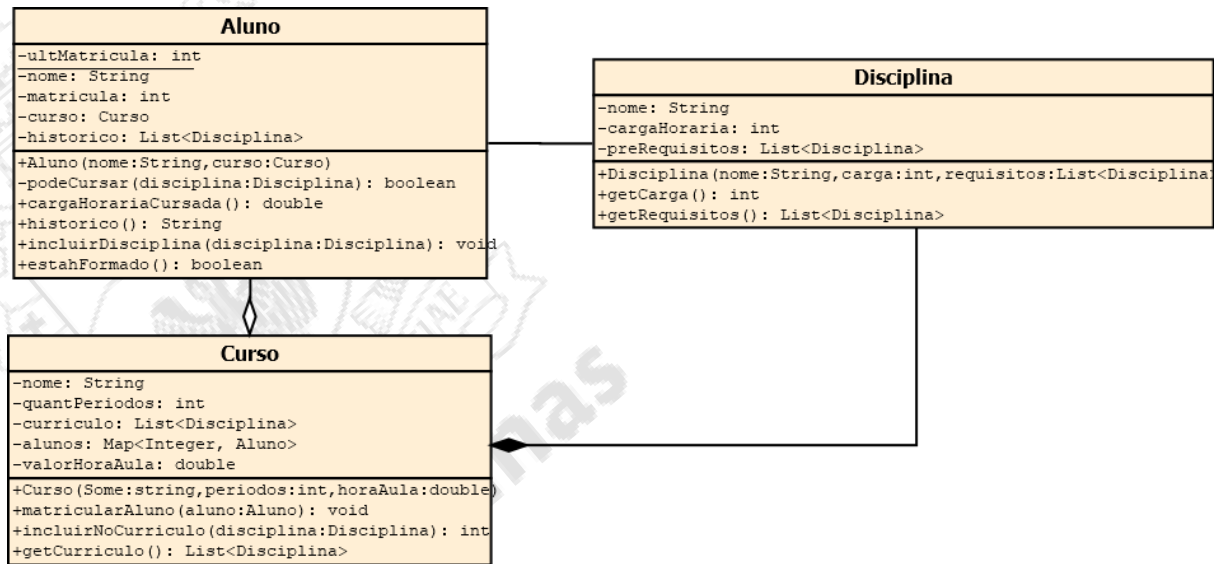
40

- ▣ P00, segundo Alan Kay
 - ▣ troca de mensagens
 - ▣ proteção e retenção local e ocultamento do estado e do processo

O SISTEMA E OS RELACIONAMENTOS

41

- Como fazer?
 - matricular um aluno no curso?
 - verificar a formatura de um aluno?
 - calcular a carga cursada pelo aluno?
 - cadastrar uma disciplina?



OBRIGADO.

DÚVIDAS?

43

BÔNUS: CARDINALIDADES

QUANTOS OBJETOS DE CADA LADO?

CARDINALIDADES

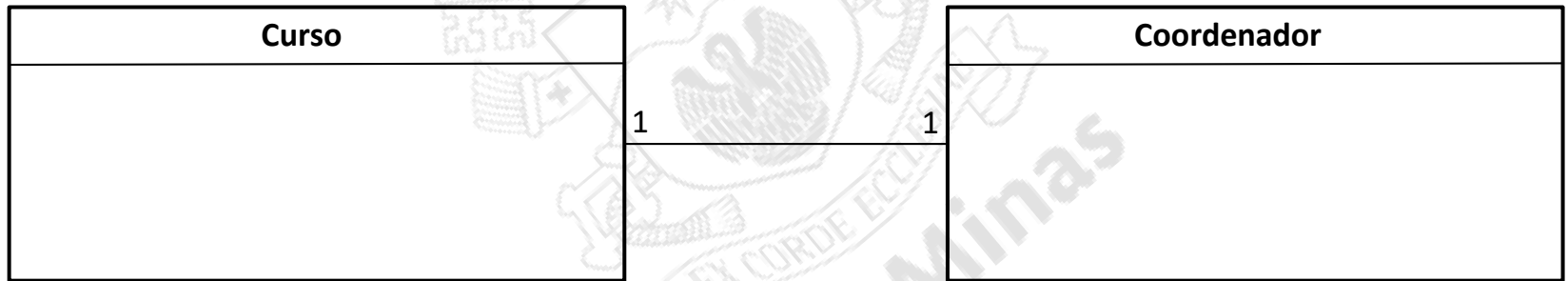
44

- Indicam a quantidade de objetos possíveis em cada lado da relação:
 - Um para um;
 - Um para muitos;
 - Muitos para muitos.

CARDINALIDADE UM PARA UM

45

- Um objeto em cada lado.
 - Ex: um curso tem um coordenador.



CARDINALIDADE UM PARA MUITOS

46

- Um dos lados pode ter múltiplos objetos.
 - Ex: um departamento possui muitos professores, mas um professor está alocado a um departamento apenas.



CARDINALIDADE MUITOS PARA MUITOS

47

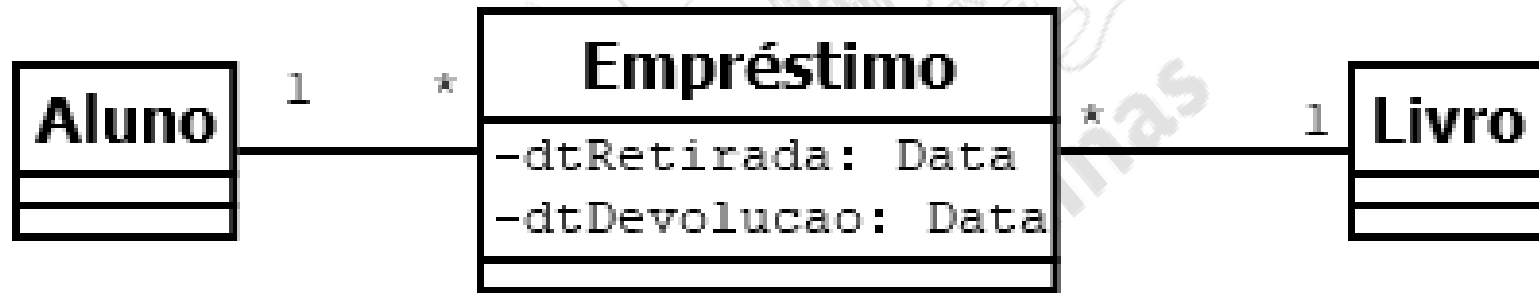
- Pode haver múltiplos objetos em ambos os lados:
 - Ex: um aluno pode realizar empréstimos de vários livros. Cada livro pode, a seu tempo, ser emprestado para vários alunos.



CARDINALIDADE MUITOS PARA MUITOS

48

- ▣ Pode gerar uma classe de associação.
 - ▣ Ex: Empréstimo.



CARDINALIDADE NA UML

49

- ▣ 1 exatamente 1
- ▣ 0..1 zero até/ou 1
- ▣ * zero ou mais
- ▣ 1..* um ou mais
- ▣ 1..10 um até dez
- ▣ 1,10 um ou dez
- ▣ 0,5..10 zero ou de cinco a dez